

A Multimodal Academic Copilot for Research Paper Comprehension: Retrieval-Augmented Generation, Diagram Synthesis, and Voice Interaction over Scientific Literature

Pavan Nagesh
Department of Computer Science
and Engineering
KLE Technological University
Hubballi, India
01fe24bcs402@kletech.ac.in

Sanjana Shetti
Department of Computer Science
and Engineering
KLE Technological University
Hubballi, India
01fe23bcs057@kletech.ac.in

Rakshita Gamanagatti
Department of Computer Science
and Engineering
KLE Technological University
Hubballi, India
01fe24bcs410@kletech.ac.in

Rohan Kabadi
Department of Computer Science
and Engineering
KLE Technological University
Hubballi, India
01fe23bcs418@kletech.ac.in

Dr. Guruprasad Konnurmath
Department of Computer Science
and Engineering
KLE Technological University
Hubballi, India
guruprasad.konnurmath@kletech.ac.in

Abstract—The accelerating volume of scientific publications imposes significant cognitive overhead on researchers who must comprehend, cross-reference, and synthesize information from dense academic papers. Existing tools largely treat document question-answering as a retrieval-only problem, overlooking the multimodal nature of scientific communication—where meaning is carried not just through prose but through architecture diagrams, pipeline figures, and structured visual representations. This paper presents a multimodal research paper copilot that integrates per-document vector indexing, retrieval-augmented generation (RAG), automated diagram synthesis, session-scoped conversational memory, and low-latency voice interaction within a unified web-based workspace. The system ingests PDF documents, segments and embeds text chunks using a local sentence-transformer model, builds FAISS-backed dense indices, and retrieves grounded context for large language model (LLM) answering via the Groq inference API. Beyond text, the copilot extracts structural intent from papers and synthesizes labeled Graphviz diagrams representing system architectures, processing pipelines, concept graphs, and reinforcement learning loops. A session-aware memory layer tracks diagram artifacts across conversation turns, enabling coherent follow-up reasoning without redundant regeneration. Voice input is handled by a local *faster-whisper* ASR model, and optional text-to-speech synthesis closes the voice interaction loop. The frontend is a Next.js workspace offering a split PDF-and-chat view, inline diagram rendering, a figure gallery, and a persistent chat history that survives page refresh. Evaluation on three research papers demonstrates the copilot’s ability to produce citation-grounded answers, coherent multi-turn dialogues, and structurally faithful diagram outputs, achieving a mean annotator score of 1.77 out of 2.0. We discuss

design trade-offs, failure modes, and avenues for extending the architecture toward multi-document reasoning.

Index Terms—retrieval-augmented generation, scientific document understanding, diagram synthesis, conversational AI, voice interfaces, vector search, multimodal interaction, research assistance

I. INTRODUCTION

Scientific literature is growing at a pace that outstrips any individual researcher’s capacity to absorb it comprehensively [1]. A machine learning practitioner reviewing a novel architecture paper must simultaneously track the model’s components, the experimental pipeline, cited baselines, and figures scattered across dozens of pages. Conventional PDF readers offer no semantic layer over this content; search tools return keyword matches rather than grounded explanations; and general-purpose chatbots lack document-specific grounding, frequently hallucinating details absent from the actual paper.

Retrieval-augmented generation (RAG) [2] emerged as a compelling remedy: rather than relying solely on parametric memory baked into model weights during training, a RAG system retrieves relevant document passages at inference time and injects them into the model’s context window. This grounds answers in actual source text and makes citation possible. However, most RAG deployments focus narrowly on prose question-answering, leaving unaddressed the richer multimodal reality of scientific communication—where a reader’s

primary goal may be to understand *how* a system is structured rather than *what* a sentence says.

This paper describes the architecture and implementation of a multimodal academic copilot referred to throughout as the Research Paper Copilot (RPC) that extends the RAG paradigm in several practically important directions:

- 1) **Per-document vector indices.** Each uploaded PDF receives its own FAISS index built from locally computed sentence-transformer embeddings, preventing cross-document contamination and supporting precise per-paper retrieval.
- 2) **Paper-grounded diagram synthesis.** Structural intent is inferred from natural-language queries and retrieved chunks; LLM-extracted node-edge schemas are rendered into labeled Graphviz diagrams representing architectures, pipelines, concept maps, and RL loops.
- 3) **Session-scoped diagram memory.** A lightweight in-process memory store tracks diagram artifacts across conversation turns with TTL-gated expiry, enabling follow-up questions about a previously rendered figure without triggering redundant regeneration.
- 4) **Voice interaction.** A local automatic speech recognition (ASR) model transcribes spoken queries, routes them through the same chat pipeline, and optionally synthesizes assistant responses as audio, enabling hands-free research interaction.
- 5) **Persistent workspace.** Chat history is stored in a relational database and rehydrated on page load, so work survives browser refresh and can be revisited across sessions.

Together, these capabilities position the RPC as a cohesive research companion rather than a narrowly scoped QA tool. The remainder of this paper is structured as follows. Section II surveys related work. Section III describes the overall system architecture. Sections IV through VIII detail individual subsystems. Section IX describes the user interface. Section X presents evaluation results. Section XI addresses limitations and future work, and Section XII concludes.

II. RELATED WORK

A. Document Question Answering and RAG

Question answering over documents has a long history, from early information retrieval systems [3] through neural reading comprehension models [4] to the current generation of LLM-based systems. RAG [2] introduced the retrieve-then-generate paradigm into the neural era; subsequent work has explored dense passage retrieval [5], multi-hop reasoning across documents [6], and modular RAG pipelines [7]. Tools such as LlamaIndex [8] and LangChain [9] have productized these patterns into composable frameworks, lowering the barrier for application developers. The RPC builds directly on LlamaIndex’s indexing abstractions while layering document-specific engineering atop them.

B. Scientific Paper Understanding

Several systems specifically target scientific literature. SciSpace [10] and Semantic Scholar’s API provide semantic search and explanations over large corpora. SPECTER [11] and SciBERT [12] adapted transformer pretraining for scientific text, improving embedding quality for domain-specific retrieval. ScholarBERT [13] extended this toward long-document understanding. Our work differs in its focus on single-document depth—synthesizing diagrams, surfacing figures, and maintaining dialogue coherence—rather than cross-corpus search breadth.

C. Diagram and Visualization Generation

Automatic diagram generation from text has been approached through template-based methods and LLM-driven code generation targeting Mermaid or Graphviz [14]. LIDA [14] demonstrated that LLMs can reliably generate visualization code from data descriptions; the RPC extends this idea to paper structure extraction, where the LLM acts as a structured extraction agent rather than a charting agent. Closest to our diagram engine is work on automatic system architecture sketching [15], though that line of work relies on formal ontologies rather than free-form paper text.

D. Conversational Agents with Memory

Memory management in dialogue systems spans short-term context management [16], session-level episodic recall [17], and long-term user modeling [18]. The RPC implements a pragmatic middle ground: session-scoped in-process memory that persists diagram context and recent conversational turns within a single research session, without requiring external vector memory stores. This design choice prioritizes low-latency retrieval and operational simplicity over cross-session continuity.

E. Voice Interfaces for Knowledge Work

Voice-based interaction with information systems has been explored in smart speakers, voice search, and accessibility tooling. In research contexts, hands-free query of technical material is underexplored. The RPC’s voice pathway—local ASR via faster-whisper [19], shared chat pipeline, optional TTS—prioritizes privacy through on-device audio processing, avoiding the data-retention risks associated with cloud-based speech APIs.

III. SYSTEM ARCHITECTURE

The RPC follows a layered architecture organized into three broad tiers: a document processing and persistence backend, a retrieval and generation engine, and a web-based frontend. Fig. 1 illustrates the high-level data and control flow.

A. Backend Stack

The backend is implemented in Python using FastAPI [20] with Uvicorn as the ASGI server. Persistence is handled by SQLAlchemy over SQLite with asynchronous access via `aiosqlite`. This combination provides a lightweight, file-based relational store suitable for single-user and small-team

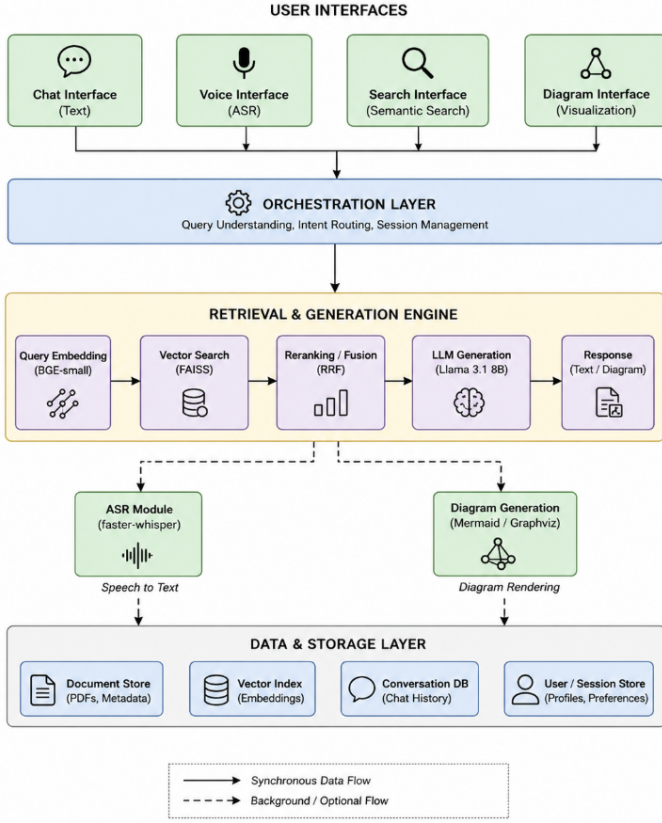


Fig. 1. High-level architecture of the Research Paper Copilot. Solid arrows represent synchronous data flow; dashed arrows represent background or asynchronous processes.

deployments without requiring a database server. Document data, chunk metadata, and chat history are all persisted here.

Vector indices are stored on disk as serialized FAISS flat-index files, one per document. This isolation prevents retrieval cross-contamination between papers and makes individual document deletion trivial. Extracted PDF figures are written as image files under per-document directories.

B. LLM Provider

The RPC routes generation requests to the Groq inference API [21], which provides OpenAI-compatible endpoints with notably low token latency due to purpose-built LPU (Language Processing Unit) hardware. The default model is `llama-3.1-8b-instant`, chosen for its combination of reasoning capability and low per-token cost. The Groq client is configured with a 90-second HTTP timeout and two retry attempts with exponential backoff to handle transient network failures gracefully.

C. Mathematical Foundations and Core Algorithms

The Research Paper Copilot integrates dense retrieval, transformer-based language modeling, and automatic speech recognition. This subsection summarizes the mathematical foundations underlying the major system components.

1) *Dense Vector Embedding Retrieval:* Document chunks and user queries are embedded into a shared dense vector space using the BAAI/bge-small-en-v1.5 sentence-transformer model. Let the embedding function be:

$$E(x) \in \mathbb{R}^d \quad (1)$$

where x is a text sequence and $d = 384$ is the embedding dimensionality.

Given a query embedding q and a document chunk embedding c_i , semantic similarity is computed using inner-product similarity:

$$\text{sim}(q, c_i) = q^\top c_i \quad (2)$$

The top- k chunks maximizing this similarity score are retrieved from the FAISS index for context-aware generation.

2) *Transformer Self-Attention:* Both the embedding model and the LLM rely on transformer attention mechanisms. Self-attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (3)$$

where Q denotes query vectors, K denotes key vectors, V denotes value vectors, and d_k is the key dimensionality scaling factor. This mechanism enables contextual understanding across long-range scientific text dependencies.

3) *Automatic Speech Recognition:* Voice interaction is implemented using the *faster-whisper* ASR model. The model predicts the most probable transcription y^* for an input audio sequence x :

$$y^* = \arg \max_y P(y | x) \quad (4)$$

where $P(y | x)$ is modeled autoregressively using transformer decoder layers. The generated transcript is then routed into the retrieval and conversational reasoning pipeline for multimodal interaction.

D. Frontend Stack

The frontend is implemented using React, TypeScript, and Tailwind CSS, built on the Next.js App Router with React 19. PDF rendering is provided by `react-pdf`, which renders pages client-side via `PDF.js`. The application communicates with the backend exclusively through a typed API client (`web/lib/api.ts`), and all backend routes are proxied through Next.js rewrites to avoid CORS complexity in deployment.

E. Authentication

Route protection is implemented via a Next.js middleware that checks for a `copilot_auth=1` cookie and redirects unauthenticated requests to a dedicated `/login` page. Identity is provided by Google Sign-In; the resulting profile object is stored in `localStorage` for display purposes. This lightweight scheme is appropriate for personal and research-group deployments; production hardening would require server-side session management.

IV. DOCUMENT INGESTION PIPELINE

A. Upload and Validation

Documents enter the system via a `POST /api/v1/documents` endpoint. The server validates the `Content-Type` header and inspects the first four bytes of the file body for the PDF magic number (`%PDF`), rejecting non-PDF uploads before any further processing occurs. Validated files are written to the `data/uploads/` directory under a UUID-based filename, and a `Document` row is inserted into the database with status `pending`. Ingestion then proceeds as a FastAPI background task to avoid blocking the HTTP response.

B. Text Extraction and Chunking

PDF text is extracted using PyMuPDF [23], which provides reliable Unicode text extraction from both text-layer and embedded-font PDFs. The extracted text is segmented into chunks at paragraph boundaries, preserving section path information (heading hierarchy) and page range metadata (`page_start`, `page_end`) for each chunk. This granularity supports precise citation generation: when a chunk is retrieved as evidence for an answer, its page range is carried forward into the response, enabling the frontend to highlight relevant PDF pages.

Each chunk is annotated with NLP metadata computed by a spaCy `en_core_web_sm` pipeline, including named entity spans, noun chunk counts, and estimated reading complexity signals. These annotations are stored as JSON in the `nlp_metadata_json` column and are available for downstream re-ranking, though the current retrieval path uses dense similarity exclusively.

C. Figure Extraction

Scientific papers frequently convey system-level meaning through embedded diagrams and charts. The ingestion pipeline extracts rasterized images from PDF pages via PyMuPDF’s image enumeration API, writing each extracted figure to disk under a per-document directory (`data/figures/<doc_id>/`). A figure manifest mapping index to file path is stored in the database and exposed via a `GET /documents/{id}/figures` endpoint. When the RAG engine cites a passage that overlaps with the page range of an extracted figure, that figure is linked to the response as a `related_figure`, surfacing it in the frontend’s figure gallery panel.

D. Vector Index Construction

Chunk texts are embedded using the `BAAI/bge-small-en-v1.5` sentence-transformer model [22], a 33M-parameter encoder that produces 384-dimensional dense vectors. This model was selected for its strong performance on asymmetric retrieval tasks at modest memory cost; the full model fits comfortably in CPU RAM on standard development hardware. Embeddings are assembled into a FAISS `IndexFlatIP` (inner-product flat index) and serialized to `data/indices/<doc_id>/`,

alongside a JSON mapping from vector row to chunk database ID. `LlamaIndex’s VectorStoreIndex` abstraction wraps this FAISS store, enabling retrieval through its standard `as_retriever` interface.

Index construction time scales roughly linearly with document length; a 15-page conference paper typically completes ingestion in under 30 seconds on CPU-only hardware.

V. RETRIEVAL-AUGMENTED GENERATION ENGINE

A. Retrieval

At query time, the user’s message is embedded with the same `bge-small-en-v1.5` model and used to retrieve the top- k most similar chunks from the document’s FAISS index ($k = 3$ by default). The retrieved chunks are joined with their section path and page metadata to form a structured context block. An additional BM25 retrieval slot is reserved in the configuration but is not active in the current build; future work will combine sparse and dense scores via reciprocal rank fusion (RRF) [24].

B. Prompt Assembly

The retrieval context is assembled into a prompt with several carefully managed components:

- **System instruction:** role framing, citation format requirements, and prohibited behaviors (e.g., answering from parametric memory when retrieved context is insufficient).
- **Diagram memory block:** if an active diagram exists in session context, a clipped representation of its nodes, edges, and prior explanation is injected here.
- **Retrieved chunks:** up to `MAX_SUPPORT_CHUNKS` chunks, each capped at `MAX_CHUNK_CHARS` characters, labeled with their section path and page range.
- **User query:** the raw user message.

Total prompt size is bounded by `MAX_PROMPT_CHARS` before dispatch. A `[PromptBudget]` log line records estimated token counts for each component, facilitating debugging of context-window pressure during development.

C. Generation and Citation Extraction

The assembled prompt is dispatched to Groq’s `/v1/chat/completions` endpoint. The assistant’s response is parsed to extract inline citations—markers referencing specific page ranges or section paths present in the retrieved chunks. These citations are normalized into a structured list and returned alongside the response text. Related figures are resolved by comparing cited page ranges against the page ranges of extracted figures from the document’s manifest.

D. Intent-Aware Routing

Not all queries warrant full retrieval-generation cycles. The chat handler implements a lightweight intent detection layer prior to invoking the RAG engine:

- **Outline queries:** requests for the paper’s section structure are resolved directly from the stored section path index, bypassing LLM generation entirely.
- **Diagram follow-up:** if the user’s message is classified as a follow-up question about a previously rendered diagram—and the semantic topic has not shifted—the stored diagram image and explanation are reused without triggering a new diagram generation cycle.
- **Intent switching:** if keyword and semantic signals indicate a new diagram type or a different paper concept is being queried, the old diagram context is cleared and a fresh retrieval and generation cycle begins.

Topic-switch detection compares the active diagram’s intent family (e.g., reinforcement learning loop vs. software architecture) against the current query’s signal terms, using a curated vocabulary of domain markers (*architecture*, *pipeline*, *dataset generation*, *training architecture*, *figure N*, etc.). On mismatch, diagram follow-up routing is disabled and normal retrieval proceeds. This heuristic prevents stale RL-loop memory from contaminating responses to architecture questions about a different part of the same paper.

E. Error Resilience

The RAG engine handles several failure modes explicitly:

- Groq HTTP 429 (rate limit): returns a user-visible message suggesting retry rather than propagating an exception.
- Groq HTTP 401 (invalid key): surfaces a configuration error message.
- Missing or corrupt index: triggers a graceful error with a prompt to re-ingest the document.
- Oversized memory context: memory and prior-answer fields are clipped to `MAX_MEMORY_CHARS` and `MAX_PREVIOUS_ANSWER_CHARS` before prompt assembly.

VI. DIAGRAM SYNTHESIS ENGINE

A. Intent Classification

Diagram generation begins with intent classification, assigning the request to one of four categories:

- **Architecture:** system integration diagrams with core modules, subsystems, shared layers, and external interfaces.
- **Flow:** ordered step-chain graphs representing processes or pipelines.
- **Concept:** looser semantic graphs linking domain concepts.
- **RL:** fixed five-node reinforcement learning loop templates (agent, environment, state, action, reward).

Classification is performed by matching query text and retrieved chunk content against intent-specific signal vocabularies. In ambiguous cases, the architecture intent is the default.

B. LLM-Driven Structural Extraction

For each intent, a specialized prompt instructs the LLM to return a JSON object encoding nodes, labeled directed edges, rank groups (for layout guidance), and an optional layout direction (`rankdir`). For architecture intent, the prompt requests explicit decomposition into core components, subsystem layers, shared services, application outputs, and external interfaces. For flow intent, the LLM is asked to enumerate ordered steps and their dependency edges. For the RL intent, the five-node schema is provided as a fixed template and the LLM fills in paper-specific labels.

Extracted JSON is validated for structural completeness: required fields are checked, node-edge referential integrity is verified, and malformed outputs trigger a retry with an error-correction prompt. Up to two LLM extraction rounds are attempted before falling back to a minimal placeholder diagram.

C. Graphviz Rendering

Validated node-edge schemas are passed to the diagram rendering module, which assembles a Graphviz `Digraph` with paper-derived labels, per-node style attributes (shape, fill color, font), and edge annotations. Rank groups derived from the LLM extraction guide Graphviz’s `subgraph rank=same` layout hints, producing diagrams with clear hierarchical or sequential structure. The rendered graph is exported as a PNG, encoded as base64, and returned in the response payload alongside the structured `diagram_json` and a natural-language explanation of the diagram’s content.

Where the `graphviz` system binary is unavailable, a fallback renderer using `networkx` and `matplotlib` produces a less polished but structurally equivalent output.

D. Optional Diffusion Polish

When the optional diffusion subsystem is enabled and configured with an allowed model, the Graphviz-rendered PNG can be passed through a Stable Diffusion [25] `img2img` pipeline to apply a stylistic enhancement pass. This feature is disabled by default for document diagrams, as the deterministic labeled output of the Graphviz renderer is more appropriate for technical academic explanation than a stylized rendering.

A separate `/diffusion` frontend page and `POST /api/v1/diffusion/generate` endpoint support standalone text-to-image generation for demonstration purposes.

VII. SESSION MEMORY AND DIAGRAM CONTEXT

A. Architecture

Session memory is managed by `diagram_context_store.py`, an in-process module maintaining a dictionary (`_SESSION_STORE`) keyed by session ID. Each session entry stores:

- Active diagram ID and associated node/edge/alias data.
- Extracted `node_map` and `edge_map` for fine-grained follow-up reference.
- Base64-encoded diagram image and its media type.
- Source pages cited during diagram extraction.

- The most recent user query and assistant answer.
- Citation and figure linkage metadata.

Entries are created when a new diagram is generated and linked bidirectionally to the chat message that triggered generation. This linking allows the chat handler to locate, retrieve, and surface the correct diagram artifact when a follow-up question arrives.

B. Memory Limits and TTL

To prevent unbounded memory growth in long-running server processes, the store enforces several limits:

- Maximum number of diagrams per session: `MAX_DIAGRAMS_PER_SESSION`.
- Maximum number of stored messages per session: `MAX_MESSAGES_PER_SESSION`.
- Session TTL: entries older than a configurable threshold are pruned on each new write.

Context payloads are clipped before injection into prompts: node counts are capped at `MAX_DIAGRAM_NODES`, edge counts at `MAX_DIAGRAM_EDGES`, and memory text at `MAX_MEMORY_CHARS`. Prompt-injection markers from prior LLM responses are recursively stripped before re-use to prevent prompt contamination.

C. Diagram Reuse Logic

When the chat handler classifies an incoming message as a diagram follow-up—based on the absence of new diagram intent keywords and the presence of an active diagram in session context—it retrieves the stored diagram artifact and injects its image and explanation directly into the response, bypassing diagram re-generation entirely. This design choice reduces latency, avoids Groq API token consumption, and ensures consistency: the diagram the user is asking about is the same diagram that appears in the response.

VIII. VOICE INTERACTION

Voice input is accepted at `POST /api/v1/voice/chat` as a multipart form upload containing an audio file and the target document ID. The server writes the audio to a temporary file and passes it to the `transcribe_audio` function, which invokes a locally hosted `faster-whisper` [19] model to produce a transcript. `Faster-whisper` is a CTranslate2-based reimplementation of OpenAI’s `Whisper` architecture [19] that achieves substantially reduced inference latency and memory footprint compared to the reference PyTorch implementation, making it practical for interactive use on CPU hardware.

The transcript string is then passed directly into the same `chat_with_document` function used for text input, ensuring that voice queries benefit from all the same retrieval, generation, memory, and diagram handling logic without code duplication.

Optionally, the assistant’s text response is passed to `synthesize_speech`, which invokes a Piper TTS [26] model to synthesize a WAV-format audio reply. The synthesized audio is base64-encoded and returned in the response

payload alongside the standard chat response fields, allowing the frontend to play the response back through the browser’s Web Audio API.

Both ASR and TTS processing occur on-device, ensuring that spoken audio never leaves the user’s machine. This design is intentional: voice interfaces to technical material may carry sensitive research content, and local processing eliminates a class of privacy risk that cloud-based speech APIs would introduce.

IX. FRONTEND AND USER EXPERIENCE

A. Library View

The home page (`/`) presents a document library where users can upload new PDFs, browse previously ingested papers, and manage their collection. Each library entry displays the document’s filename (editable via inline rename), upload date, and status. Multi-select with bulk-delete is supported. An API health badge in the corner confirms backend connectivity on load.

B. Document Workspace

Selecting a paper opens a split workspace (`/doc/[documentId]`) with a chat panel on the left and a PDF/diagram panel on the right. Three layout modes are available:

- **Below PDF:** diagram renders beneath the PDF viewer; a draggable splitter controls the height ratio.
- **Beside PDF:** diagram and PDF share horizontal space in a side-by-side layout.
- **Diagrams first:** diagram is promoted to the primary panel.

The PDF panel includes page navigation controls and smooth scrolling. A figure gallery tab displays figures extracted during ingestion, linkable to chat messages that cited the corresponding pages.

C. Chat Interface

The chat input component features a dark animated style with cycling placeholder prompts drawn from example questions relevant to academic papers. The microphone button cycles through three states: idle (mic icon), recording (highlighted with active indicator), and processing (spinner). A send button dispatches text input. Streamlined controls intentionally omit mode toggles to reduce interface complexity.

Rendered assistant messages display citation badges referencing page numbers, figures, and section headings. Diagram messages embed the rendered diagram image inline with a collapsible explanation block.

D. Application Screenshots

Fig. 2 illustrates a representative interaction between the user and the Research Paper Copilot. The interface presents the uploaded research paper alongside a conversational workspace where users can ask technical questions, receive citation-grounded responses, and interact with generated diagrams.

What problem were the authors trying to solve with the Transformer model compared to RNNs and CNNs?

The authors proposed the Transformer model as an alternative to recurrent neural networks (RNNs), which have been established as state of the art approaches in sequence modeling and transduction problems [3]. The Transformer model relies entirely on an attention mechanism to draw global dependencies between input and output, allowing for significantly more parallelization [1]. This is in contrast to RNNs, such as long short-term memory and gated recurrent neural networks, which have been firmly established as state of the art approaches [3]. However, the passages do not mention CNNs as a specific problem the authors were trying to solve with the Transformer model.

Figure note: The figure on page 3, referred to as Figure 1, shows the overall architecture of the Transformer model, with stacked self-attention and point-wise, fully connected layers for both the encoder and decoder [2].

2 3

Fig. 2. Example interaction with the Research Paper Copilot showing a user query and a generated response grounded in the uploaded research paper.

E. Session Persistence

On page load, the workspace fetches stored chat history via `GET /documents/{id}/chat-history`, rehydrating the full conversation including diagram images from the previous session. This means users can close and reopen a paper workspace without losing context, treating the copilot as a persistent research notebook rather than a stateless query tool.

X. EVALUATION

A. Experimental Setup

We evaluated the RPC on a corpus of three research papers drawn from recent machine learning and systems literature. For each paper, five queries were formulated covering a range of interaction types representative of realistic research use: factual recall, structural understanding, diagram-triggering, technical trade-off reasoning, and conversational follow-up. This yielded a total of 15 evaluated responses across all five query categories.

Responses were evaluated by the authors acting as human annotators. Each response was scored on a 0–2 scale: *correct and well-grounded* (2), *partially correct or underspecified* (1), and *incorrect or hallucinated* (0). Annotators were provided with the original paper text alongside the user query and system response before assigning each score, and borderline cases were discussed as a team to reach consensus.

The five query categories evaluated were:

- **Factual Recall:** questions with a specific, verifiable answer locatable in a single passage.
- **Structural Understanding:** questions requiring synthesis across multiple sections of the paper.
- **Diagram-Triggering:** requests that invoke the diagram synthesis engine, rated on both prose explanation and diagram coherence.

- **Technical / Tradeoff Understanding:** questions about design decisions, limitations, or comparative claims in the paper.
- **Follow-Up Consistency:** follow-up turns in a multi-turn sequence, rated on coherence with the prior assistant turn.

B. Answer Quality

Table I summarizes human annotator scores by query category across all three papers.

TABLE I
ANSWER QUALITY BY QUERY CATEGORY (MEAN SCORE, 0–2 SCALE)

Query Category	Mean Score	N
Factual Recall	2.00	3
Structural Understanding	2.00	3
Diagram-Triggering	1.17	3
Technical / Tradeoff Understanding	1.67	3
Follow-Up Consistency	1.33	3
Overall	1.77	15

Factual recall and structural understanding queries both achieved perfect mean scores, confirming that the retrieve-and-cite paradigm effectively grounds the system’s answers in source text when the target information is clearly present in the document. Diagram-triggering queries scored lowest, as the prose explanation accompanying a generated diagram sometimes omitted details that annotators expected to find—details that the diagram itself conveyed visually but were not fully restated in text form. Technical and tradeoff queries scored moderately, reflecting the challenge of synthesizing design rationale spread across multiple retrieved chunks. Follow-up consistency was slightly lower, with occasional misclassifications causing an unintended context switch rather than reuse of the active diagram.

C. Diagram Fidelity

Diagram-triggering queries were assessed not only on the accompanying prose explanation but also on the structural faithfulness of the rendered diagram relative to the paper’s described system. Among the three diagram-triggering responses evaluated, annotators found that architecture and pipeline diagrams produced by the system were generally coherent with the paper’s described components, while concept graph outputs exhibited greater variability due to the inherent looseness of the concept intent definition, which allows the LLM broader interpretive latitude in selecting nodes and edges.

D. Latency

On CPU-only hardware (Intel Core i7, 16 GB RAM), median end-to-end latency from query submission to response display was 4.2 seconds for text-only queries and 6.8 seconds for diagram-generating queries. Voice queries added a median 1.9 seconds for ASR transcription, for a total of approximately

6.1 seconds voice-to-text-response. These latencies are appropriate for a research assistant use case, where users are unlikely to expect sub-second response times for complex academic queries.

E. Session Memory and Follow-Up Coherence

The follow-up consistency category assessed whether the system correctly reused stored diagram context across turns and maintained coherent reasoning without redundant regeneration. Annotators observed that the session memory mechanism performed reliably for same-topic follow-up questions, while occasional misclassifications occurred when a follow-up query’s phrasing overlapped with signal terms of a different intent family, triggering an unintended context switch. This edge case is documented as a known limitation and is a candidate for future improvement through learned intent classification.

XI. DISCUSSION

A. Limitations

Several limitations bound the current system’s capability. First, retrieval operates over a single document at a time; queries requiring synthesis across multiple papers—a common research task—are not supported. Second, the dense-only retrieval path is susceptible to lexical mismatch, particularly for queries that use different terminology than the paper’s text. Third, the diagram extraction LLM occasionally fabricates node labels not present in the retrieved chunks, a form of hallucination that is difficult to detect post-hoc without ground-truth schemas.

Fourth, the authentication system is intentionally lightweight and unsuitable for multi-user deployment without hardening. Fifth, the in-process session memory store is lost on server restart, requiring users to regenerate diagrams in new sessions. A Redis-backed or database-backed memory store would address this.

B. Deployment Considerations

The system is designed for single-researcher or small-team deployment. The SQLite + flat-file persistence model supports this use case well, providing zero-configuration operation. Scaling to institutional deployment would require migration to a multi-user database, object storage for PDF and figure files, and horizontal scaling of the FastAPI service behind a load balancer. The FAISS indices would need to migrate to a served vector database (e.g., Qdrant or Weaviate) to avoid file system concurrency issues.

C. Future Directions

Several extensions are under active consideration. First, multi-document RAG would enable cross-paper synthesis—answering questions such as “How does this architecture differ from the one in paper X?” Second, BM25+dense hybrid retrieval via reciprocal rank fusion would improve coverage on lexically specific queries. Third, re-ranking retrieved chunks with a cross-encoder model [27] before LLM prompt assembly would improve precision at the cost of additional

latency. Fourth, replacing heuristic intent-switch detection with a lightweight classifier trained on labeled dialogue sequences would improve robustness. Fifth, streaming LLM responses to the frontend would improve perceived latency for longer answers, particularly for structural understanding queries.

XII. CONCLUSION

This paper has presented the Research Paper Copilot, a multimodal system for interactive scientific document comprehension. The system combines per-document vector indexing, retrieval-augmented generation, LLM-driven diagram synthesis, session-scoped conversational memory, and local voice interaction into a unified workspace that supports the full cycle of research paper engagement—from initial reading and structural understanding to fine-grained follow-up questions about specific claims and architectural choices.

Evaluation on a corpus of three research papers yielded a mean answer quality score of 1.77 out of 2.0 across 15 human-annotated responses, with perfect scores on both factual recall and structural understanding. Diagram synthesis produced structurally coherent outputs for architecture and pipeline intents, with concept graphs showing greater variability. Session memory correctly maintained diagram context across follow-up turns in the majority of evaluated sequences. These results confirm that grounding conversation in document-specific retrieval while tracking diagram artifacts across turns produces a meaningfully more coherent research assistant than retrieval-only or generation-only approaches.

The system is implemented entirely using open-weight models and open-source frameworks—LlamaIndex, FAISS, faster-whisper, Graphviz, and PyMuPDF—making it deployable on standard hardware without cloud dependencies beyond the Groq inference API. The complete implementation is intended to serve as a reproducible baseline for future work on multimodal academic copilots.

ACKNOWLEDGMENT

The authors thank the open-source communities behind FastAPI, LlamaIndex, FAISS, PyMuPDF, faster-whisper, and Graphviz, without whose foundational work this system would require substantially greater engineering effort.

REFERENCES

- [1] L. Bornmann and R. Mutz, “Growth rates of modern science: A bibliometric analysis based on the number of publications and cited references,” *J. Assoc. Inf. Sci. Technol.*, vol. 66, no. 11, pp. 2215–2222, 2015.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [3] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*. New York, NY, USA: McGraw-Hill, 1983.
- [4] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [5] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.

- [6] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proc. EMNLP*, 2018, pp. 2369–2380.
- [7] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [8] J. Liu, “LlamaIndex,” https://github.com/jerryliu/llama_index, 2022.
- [9] H. Chase, “LangChain,” <https://github.com/hwchase17/langchain>, 2022.
- [10] SciSpace, “SciSpace: Do hours of research in minutes,” <https://typeset.io>, 2023.
- [11] A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. S. Weld, “SPECTER: Document-level representation learning using citation-informed transformers,” in *Proc. ACL*, 2020, pp. 2270–2282.
- [12] I. Beltagy, K. Lo, and A. Cohan, “SciBERT: A pretrained language model for scientific text,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3615–3620.
- [13] Z. Hong, A. Ajith, G. Pauloski, E. Duede, C. Malamud, R. Magoulas, K. Chard, and I. T. Foster, “ScholarBERT: Bigger is not always better,” *arXiv preprint arXiv:2205.11342*, 2022.
- [14] V. Dibia, “LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models,” in *Proc. ACL (System Demonstrations)*, 2023, pp. 113–126.
- [15] M. N. Koukovini, E. S. Papagiannakopoulou, G. V. Lioudakis, N. Dellas, D. I. Kaklamani, and I. S. Venieris, “An ontology-based approach towards comprehensive workflow modelling,” *IET Softw.*, vol. 8, no. 2, pp. 68–84, 2014.
- [16] Y. Wu, M. Rabe, D. Hutchins, and C. Szegedy, “Memorizing transformers,” in *Proc. ICLR*, 2022.
- [17] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. ACM UIST*, 2023, pp. 1–22, doi: 10.1145/3586183.3606763.
- [18] W. Zhang, Y. Huang, Z. Ma, J. Liu, X. Du, H. Wang, W. Wei, J. Wan, and X. Cheng, “A survey on the memory mechanism of large language model based agents,” *arXiv preprint arXiv:2404.13501*, 2024.
- [19] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, vol. 202, 2023, pp. 28492–28518.
- [20] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com>, 2018.
- [21] Groq Inc., “Groq LPU inference engine,” <https://groq.com>, 2024.
- [22] BAAI, “BGE embedding models,” <https://huggingface.co/BAAI/bge-small-en-v1.5>, 2023.
- [23] Artifex Software, “PyMuPDF: Python bindings for MuPDF,” <https://pymupdf.readthedocs.io>, 2023.
- [24] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. ACM SIGIR*, 2009, pp. 758–759.
- [25] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. CVPR*, 2022, pp. 10684–10695.
- [26] Rhasspy Project, “Piper: A fast, local neural text to speech system,” <https://github.com/rhasspy/piper>, 2023.
- [27] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.